Neural Ordinary Differential Equations: A Reduced-Scale Implementation and Experimental Study

Satyasheel Raman • 24CS3047
Rajiv Gandhi Institute of Petroleum Technology
Instructor: Dr Ankur Pandey

Abstract

Neural Ordinary Differential Equations (Neural ODEs) model hidden representations as solutions of learned differential equations rather than as a fixed sequence of layers. This report presents a reduced-scale PyTorch study inspired by Chen et al. (NeurIPS 2018). Experiments cover synthetic spiral classification, comparison of direct and adjoint differentiation, learning a known damped-rotation vector field, Runge–Kutta step-size sensitivity, a simplified Latent ODE reconstruction ablation, and MNIST classification against a residual-network baseline. In the recorded MNIST run, the Neural ODE achieved 92.96% accuracy with 22,026 parameters, while the ResNet achieved 93.64% with 46,986 parameters. A scalar linear-ODE toy experiment also matched a custom adjoint gradient to PyTorch autograd within numerical precision. The results illustrate trade-offs among model size, numerical accuracy, and runtime; they are not an exact reproduction of the paper.

Index Terms

Neural ordinary differential equations, continuous-depth networks, adjoint sensitivity, residual networks, numerical integration, Latent ODE, MNIST.

I. Introduction

Deep neural networks usually transform representations through a discrete sequence of layers. A residual network updates a hidden state by adding a learned increment, which can be interpreted as a discretized dynamical system. Neural ODEs take this connection further: the hidden state evolves continuously according to a parameterized derivative function, and a numerical ODE solver computes the state at a requested time [1].

This formulation offers solver-controlled computation and a continuous-depth view of representation learning. The adjoint sensitivity method computes gradients by solving an augmented differential equation backward in time, potentially reducing memory requirements relative to storing every intermediate solver state. Its practical cost depends on the solver, dynamics, tolerances, and implementation.

The project has four goals: (1) implement a Neural ODE classifier and compare it with a ResNet baseline; (2) examine direct versus adjoint differentiation and implement a transparent scalar adjoint toy; (3) test numerical integration and Latent ODE settings through ablations; and (4) contextualize a reduced MNIST experiment against results reported in the original paper.

II. Methodology

A. Neural ODE classifier

Let h(t) be a hidden representation. Its evolution is defined by

$$dh(t)/dt = f_\theta(h(t), t), \quad h(t_1) = h(t_0) + \int_{[t_0,t_1]} f_\theta(h(t),t)\, dt.$$

An encoder maps the input to $h(t_0)$; an ODE solver integrates the learned vector field; and a linear classification head maps the terminal state to class logits. The experiments use PyTorch and torchdiffeq for the main Neural ODE models [2].

B. Residual baseline

For the spiral task, the ResNet uses a 2→64 encoder followed by four residual MLP blocks of the form $h \leftarrow h + 0.1g(h)$, then a classification head. It has 33,602 trainable parameters, compared with 8,642 for the Neural ODE in this setup. The MNIST baseline is a separately configured residual model; its parameter count and performance are reported independently.

C. Adjoint sensitivity and toy gradient check

For terminal loss L, the adjoint $a(t)=\partial L/\partial h(t)$ satisfies a backward differential equation involving the Jacobian of f_θ. The main solver comparison uses torchdiffeq's odeint and odeint_adjoint on the same small batch and model initialization. In addition, a scalar linear system $dh/dt=ah$ is used for a transparent special-case check: the terminal-state gradient is calculated by integrating the scalar adjoint sensitivity, using a trapezoidal approximation for the parameter-gradient integral, and compared with PyTorch autograd. This is a toy special-case implementation, not a general-purpose adjoint ODE solver.

D. Known dynamics and solver ablation

A neural vector field is trained on a damped rotation with $dx/dt=ax-\omega y$ and $dy/dt=\omega x+ay$, using $a=-0.1$ and $\omega=1.5$. Derivative-prediction error and integrated trajectory error measure whether the learned field recovers the known dynamics. A fixed trained field is then evaluated with RK4 step sizes 0.20, 0.10, and 0.05 to assess the accuracy/runtime trade-off.

E. Simplified Latent ODE

The trajectory-reconstruction experiment encodes an observed sequence into a latent mean and log-variance, samples an initial latent state, evolves it through an ODE, and decodes reconstructed observations. The objective combines reconstruction error and a KL regularizer weighted

by β. Two β settings are compared; this is a simplified educational experiment rather than a full reproduction of every model and dataset in [1].

III. Experimental Setup

The reported values are the saved outputs in the revised notebook, not results from a newly completed clean-runtime rerun. The notebook uses PyTorch and torchdiffeq. Synthetic experiments use generated spiral points and a known damped-rotation system. The MNIST run uses a subset of 20,000 training examples and 5,000 test examples, normalized with mean 0.1307 and standard deviation 0.3081; both models train for five epochs with Adam. Because the dataset subset, architecture, training schedule, and compute setup differ from the original paper, absolute paper-versus-project scores are contextual only.

For methodological caution, the spiral notebook evaluates test accuracy during training. A more rigorous final run should use a separate validation set for model selection and evaluate the held-out test set only once after training. Runtime figures are single-run observations and should not be treated as stable benchmark estimates.

IV. Results

A. Spiral classification

Model	Test accuracy	Parameters	Time (s)
Neural ODE	71.88%	8,642	24.62
ResNet	69.38%	33,602	2.89

On this saved synthetic split, the Neural ODE exceeded the ResNet by 2.50 percentage points and used about 74.3% fewer parameters. It was slower in this run, emphasizing that fewer parameters do not automatically imply lower wall-clock time.

B. Direct versus adjoint differentiation

Method	Loss	Gradient norm	Time (s)
odeint	0.663166	0.645644	0.0230
odeint_adjoint	0.663166	0.645644	0.1159

The losses and gradient norms match to the displayed precision on this batch. The adjoint call took longer in this small single run. Peak memory was not measured, so the experiment does not establish a memory advantage empirically.

C. Custom scalar adjoint check

For dh/dt=ah, the terminal state was 1.702881 and the loss was 0.04413983. PyTorch autograd returned a gradient of −0.50595822; the toy custom adjoint returned −0.50595822, with absolute difference 1.110×10^{-16}. This confirms the implementation for the chosen scalar example, not for arbitrary nonlinear vector fields.

The toy check is intentionally narrow so that the derivative and sensitivity can be inspected independently. It is useful as a unit test for the sign and scaling of the adjoint calculation. A production implementation would need to handle vector states, parameter Jacobians, time-dependent dynamics, solver error control, and gradients with respect to all relevant inputs.

D. Learning known dynamics

The derivative-prediction MSE fell from 1.808793 at epoch 0 to 0.000087 at epoch 300. The integrated trajectory MSE was 0.000006, indicating that the fitted vector field closely reproduced this generated damped-rotation system.

This experiment is a controlled sanity check because the generating dynamics are known. It separates local vector-field fitting from accumulated integration behavior: a low derivative error measures local predictions, while trajectory MSE evaluates the result after repeated numerical integration. Since the data are synthetic and generated from the same simple system, this result should not be generalized to noisy real-world dynamics.

E. RK4 step-size ablation

Step size	Trajectory MSE	Time (s)
0.20	0.0000857	0.0332
0.10	0.0000102	0.0434
0.05	0.0000056	0.0667

Smaller RK4 steps reduced trajectory error for the fixed learned field, while runtime increased. The result is consistent with the expected accuracy-versus-computation trade-off for numerical integration.

F. Latent ODE β ablation

β	Clean MSE	Noisy MSE	KL
0.001	0.025113	0.025628	2.0222
0.0001	0.026189	0.026697	2.2602

The β=0.001 run had slightly lower clean and noisy reconstruction MSE and a lower KL value than β=0.0001. This comparison covers only two settings and does not establish a universally optimal β.

The KL term regularizes the latent distribution toward its prior, while the reconstruction term rewards fidelity to observations. Changing β changes the balance between these objectives, so a single metric cannot fully characterize model quality. A stronger study would evaluate several random seeds, inspect reconstructed trajectories visually, and report both reconstruction and latent regularization behavior across a wider β range.

G. MNIST classification

Model	Accuracy	Error	Parameters	Time (s)
Neural ODE	92.96%	7.04%	22,026	39.62
ResNet	93.64%	6.36%	46,986	28.20

The ResNet was 0.68 percentage points more accurate in this run. The Neural ODE used about 53.1% fewer parameters but took longer to train in the recorded run. These results apply to the stated subset and five-epoch setup.

Parameter count measures the number of learned scalar values, not the number of function evaluations or total arithmetic operations. Neural ODE runtime depends on how many solver steps are needed and on the cost of evaluating the derivative network. Consequently, the parameter advantage should be reported separately from runtime and predictive performance.

V. Context with the Original Paper

Original paper model	MNIST test error	Parameters
ResNet	0.41%	0.60M
RK-Net	0.47%	0.22M
ODE-Net	0.42%	0.22M
This study: Neural ODE	7.04%	22,026
This study: ResNet	6.36%	46,986

Chen et al. report 0.42% MNIST test error for ODE-Net (approximately 0.22M parameters) and 0.41% for ResNet (0.60M parameters) [1]. Our reduced-scale result has substantially higher error, as expected from a different model and limited training protocol. It is not valid to interpret this as a direct replication gap because the data volume, architecture, solver/training details, and compute budget are not matched. The comparison is included to anchor the project against the paper, not to claim equivalence.

VI. Discussion and Limitations

The experiments reveal several complementary findings. On spirals, the Neural ODE was more accurate with fewer parameters but slower than the residual baseline. On MNIST, the ResNet achieved higher accuracy, while the Neural ODE remained more parameter-efficient. For the learned synthetic dynamics, the vector field fit closely matched the target trajectories. Smaller RK4 steps improved numerical accuracy at a higher computational cost. The Latent ODE β experiment showed a modest reconstruction difference between the two tested settings.

Limitations include the reduced MNIST subset and short training schedule, single-run timing measurements, a narrow solver-step and β sweep, and no peak-memory measurement. The custom adjoint code is deliberately restricted to a scalar linear ODE and should not be described as a general adjoint solver. The main classifier relies on the torchdiffeq implementation. Finally, repeated use of the spiral test split during training can bias model selection; a validation split is recommended.

Future work should repeat each configuration with multiple seeds and report mean and standard deviation, separate validation from test evaluation, and profile peak memory alongside wall-clock time. A closer reproduction would also match the paper's dataset protocol, architecture, optimizer schedule, solver configuration, and evaluation details. These controls would make it possible to distinguish architectural effects from differences in experimental setup.

VII. Conclusion

This reduced-scale study implemented and evaluated Neural ODE components across classification, known-dynamics learning, numerical solver sensitivity, and latent trajectory reconstruction. The saved experiments show that continuous-depth models can offer parameter efficiency, but accuracy and runtime depend on task and configuration. The scalar adjoint check agrees with autograd for its special case, while the solver and β ablations expose useful trade-offs. A full reproduction would require matching the original architectures, datasets, training protocol, solver settings, and evaluation procedure.

References

[1] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, “Neural Ordinary Differential Equations,” in Advances in Neural Information Processing Systems 31 (NeurIPS), 2018.

[2] R. T. Q. Chen et al., “torchdiffeq: Differentiable ODE solvers with PyTorch,” software repository. Available: https://github.com/rtqichen/torchdiffeq

[3] Y. LeCun, C. Cortes, and C. J. C. Burges, “The MNIST database of handwritten digits.” Available: http://yann.lecun.com/exdb/mnist/